Software Development Life Cycle (SDLC)

90-Minute University Lecture Plan & Teaching Notes

What is Software Development?

"If a client gives us an idea for an app today, can a developer simply start writing code?" Usually, no. A professional software project requires understanding the problem, planning the work, gathering requirements, designing the solution, coding, testing, deploying, and maintaining the software.

A useful analogy is building a house. We do not normally buy bricks and immediately start building. We first understand the customer's needs, prepare a plan/design, estimate resources and cost, construct the building, inspect it, hand it over, and maintain it. Software development follows a similar structured approach.

. Definition of SDLC

Software Development Life Cycle (SDLC) is a structured process used to plan, develop, test, deploy, and maintain software systems.

Simple classroom definition: "SDLC is a step-by-step framework that guides a software project from its initial idea to development, delivery, and ongoing maintenance."

. Why Do We Need SDLC?

- To understand what the customer actually needs.
- To organize a large and complex project into manageable activities.
- To control time, cost, resources, and scope.
- To reduce errors and project risks.
- To improve software quality.
- To make responsibilities and deliverables clear.
- To provide documentation and a basis for maintenance.

Important point for students: SDLC does not mean that every project must follow exactly the same sequence. The activities are common, but different SDLC models arrange and repeat them differently.

Main Phases of SDLC

A commonly taught SDLC structure contains seven major phases:

1. 1. Planning and Feasibility
2. 2. Requirements Analysis
3. 3. System Design
4. 4. Implementation / Coding
5. 5. Testing

6. 6. Deployment
7. 7. Maintenance

Overall flow: Idea → Planning → Requirements → Design → Coding → Testing → Deployment → Maintenance

Phase 1 — Planning and Feasibility

Purpose: Decide what problem we are solving, why we should solve it, and whether the project is practical.

Typical activities:

- Identify the business or user problem.
- Define project objectives and initial scope.
- Identify stakeholders.
- Estimate time, cost, people, technology, and resources.
- Identify major risks.
- Conduct feasibility analysis.

Common feasibility questions:

- Technical feasibility: Can we build it with available technology and skills?
- Economic feasibility: Is the expected benefit worth the cost?
- Operational feasibility: Will the organization and users be able to use it effectively?
- Schedule feasibility: Can it be delivered within the required time?
- Legal/regulatory feasibility: Does it comply with applicable requirements?

Typical output: Initial project plan, feasibility report, scope statement, and high-level schedule.

Phase 2 — Requirements Analysis

Purpose: Find out exactly what the system must do and what constraints it must satisfy.

Two important categories:

- Functional requirements — describe what the system should do.

Example: “The system shall allow a student to register for an available course.”

- Non-functional requirements — describe quality attributes or constraints.

Example: “The system shall display the student's course list within 3 seconds.”

Requirement activities:

- Elicitation: collect information from stakeholders.
- Analysis: remove ambiguity, conflicts, and unnecessary requirements.
- Specification: document requirements, often in an SRS.
- Validation: check that requirements are correct, complete, consistent, and testable.
- Requirements management: control changes to requirements.

Typical output: Software Requirements Specification (SRS), use cases, user stories, acceptance criteria, and requirement traceability information.

Phase 3 — System Design

Purpose: Convert approved requirements into a technical blueprint for building the software.

Design may cover:

- System architecture.
- Database design.
- User interface design.
- Module/component design.
- API/interface design.
- Security and performance considerations.

Common UML artifacts:

- Use Case Diagram
- Class Diagram
- Activity Diagram
- Sequence Diagram
- State Diagram

Typical output: Architecture/design documents, UML diagrams, database schema, interface specifications, and prototypes where applicable.

Phase 4 — Implementation / Coding

Purpose: Transform the design into working software.

- Developers write and review source code.
- Developers follow coding standards and version control practices.
- Individual modules/components are implemented.
- Developers perform basic checks and unit tests.
- Code is integrated with other components.

Typical output: Source code, builds, unit tests, and technical documentation.

Phase 5 — Testing

Purpose: Find defects and determine whether the software meets specified requirements and quality expectations.

Common testing levels:

- Unit testing — tests individual functions/modules.

- Integration testing — tests interactions between modules.
- System testing — tests the complete integrated system.
- Acceptance testing — checks whether the system is acceptable to the customer/business.

Other concepts to introduce:

- Black-box testing
- White-box testing
- Regression testing
- Test cases and test data
- Defect/bug reporting
- Test automation

Simple example: If the requirement says a student cannot register for a full course, testing should include a full-course case and verify that registration is rejected with an appropriate message.

Phase 6 — Deployment

Purpose: Make the tested software available to its intended users.

- Prepare the production environment.
- Install/configure the application.
- Migrate data when required.
- Train users where necessary.
- Monitor the system after release.
- Use a rollback/recovery plan when appropriate.

Deployment strategies may include big-bang deployment, phased deployment, pilot deployment, and rolling/canary approaches.

Phase 7 — Maintenance

Software does not normally end when it is deployed. After release, it must be monitored, corrected, adapted, and improved.

Common maintenance categories:

- Corrective maintenance — fixes discovered defects.
- Adaptive maintenance — adapts software to changed environments, platforms, laws, or business needs.
- Perfective maintenance — improves features, usability, or performance.
- Preventive maintenance — reduces the likelihood of future problems, for example through refactoring or modernization.

Running Example — University Course Registration System

SDLC Phase	What We Do	Example Output
Planning	Identify problem:	Project scope + feasibility

	manual/inefficient registration	
Requirements	Interview students, faculty, registrar	SRS + use cases
Design	Design database, UI, architecture	UML + database schema
Coding	Build login, course search, registration modules	Working software
Testing	Test registration rules and system behavior	Test cases + defect reports
Deployment	Release system to university users	Production system
Maintenance	Fix bugs and add approved improvements	Updated releases

SDLC and SDLC Models

Clarify an important distinction: SDLC describes the overall development life-cycle activities. An SDLC model is a particular way of organizing and managing those activities.

Model	Basic Idea	Typical Characteristic
Waterfall	Phases are largely completed in sequence	Up-front requirements and planned progression
Incremental	System is delivered in multiple increments	Useful when functionality can be prioritized and delivered in parts
Spiral	Development proceeds through repeated risk-focused cycles	Strong emphasis on risk analysis
Agile	Short iterative cycles with frequent feedback	Frequent delivery, collaboration, and adaptation

.